Learning to Manipulate a New Object from Action-Free Videos

Group 9: Brik Meza, Myriam Dardour, Zheng Ye Zhang, Gonzalo Millán García
McGill University
ECSE 552 — Final report (draft)

Abstract

Visuomotor policies imitate images paired with motor commands, and every new object normally requires new paired demonstrations. We study whether videos without actions are enough to extend an image-only policy from one object to another. In simulation (OCBench, UR5e arm), labeled demonstrations exist only for a red source cube; each target object has only videos. An inverse dynamics model (IDM), trained on the source and adapted to the target with domain-adversarial training (DANN), labels the target videos, and an image-only ACT policy trains on both. In preliminary single-seed results, the source-only policy solves the source cube from images alone (99% test success, 10 ms per inference call) but fails when only the cube's colour changes (7%), which leaves a large gap for the action-free methods to close. [TODO: Results of the IDM methods against the baselines, R4 and R5, and the conclusion.]

1 Introduction

Application (C1). Warehouse and home robots meet new items constantly. A robot arm that already manipulates one object should learn to handle a new one without anyone operating the robot again. We study this for a tabletop pick-and-place task in the OCBench simulator [1], built on MuJoCo [2]: a UR5e arm with a Robotiq gripper picks up a block and places it inside a goal square, seeing the scene only through three RGB cameras (Figure 1).

Why a solution is needed (C3). Visuomotor policies such as ACT [3] and Diffusion Policy [4] imitate images paired with motor commands. Acquiring such pairs is costly because it requires operating the robot: 50 people took a year to collect the 76k teleoperated demonstrations of DROID [5]. With limited coverage, a policy that meets an unfamiliar situation may output an erroneous action, and errors compound during execution, moving the robot further from its training data [3]. Videos are far cheaper than paired data: they can be recorded without logging commands, and world models can now generate them [6]. The open question is how to turn videos into supervision for a policy that must output motor commands.

Related work. The standard recipe trains an *inverse dynamics model* (IDM) on transitions with known actions, uses it to label observation-only demonstrations, and clones the result: behavioral cloning from observation [7], scaled to internet video by VPT [8] and to video generated by world models by DreamGen [6]. Morin et al. [9] analyse why this is sample efficient with few labels and name unlabeled data from a different environment as future work; this is our setting. Kim et al. [10] label human eye-in-hand videos with an IDM trained on robot play. A second line avoids explicit pseudo-actions by learning latent actions or motion representations from videos [11–13]; these methods need larger models and data than this project allows. When the videos show a new object, the IDM is applied outside its training distribution. Unsupervised domain adaptation addresses this shift by aligning source and target features, adversarially in DANN [14] and its successors [15, 16]. Alignment has been applied to the policy's encoder for visuomotor transfer [17, 18], but Cheng

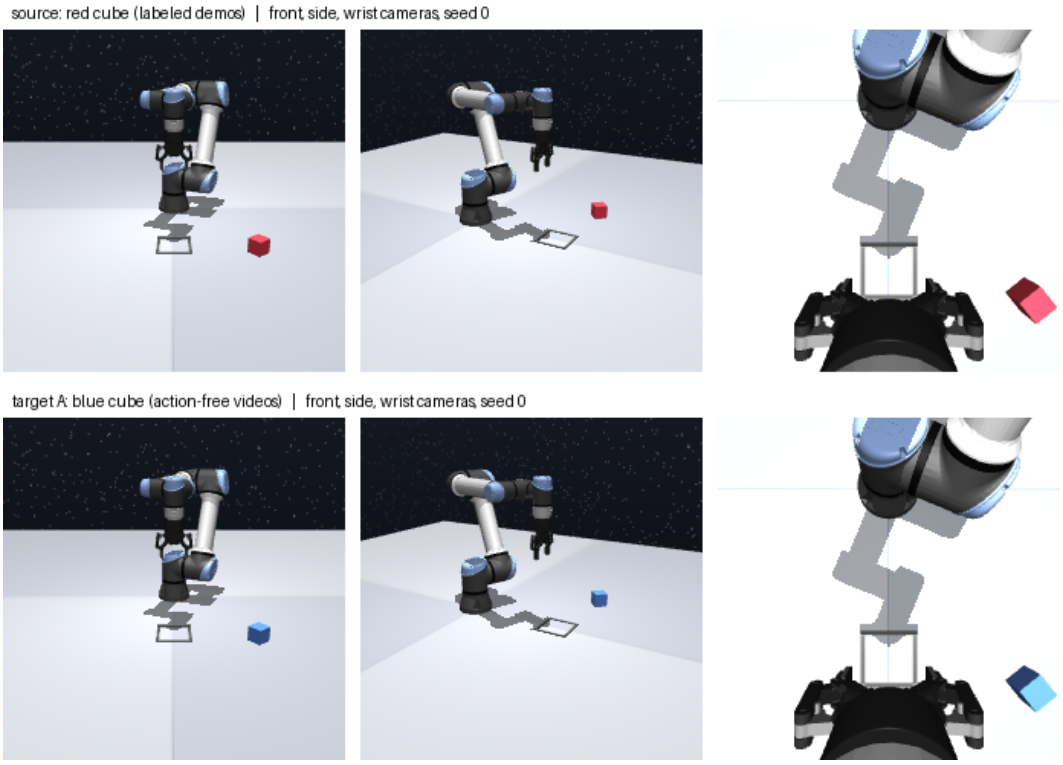


Figure 1: Initial state seen by the front, side and wrist cameras, for the red source cube and the blue target A. The gray square is the goal.

et al. [17] require action-labeled target demonstrations. Two warnings apply to our design: invariant features can hurt when the label distribution differs between domains [19], and adversarial alignment was designed for classification and is weaker for regression [20]. We found no work that adapts an IDM adversarially to pseudo-label action-free videos of a new object for the same robot; we frame this as a controlled study and not as a new method.

Problem (C2, high level). Labeled demonstrations (images and actions) exist only for a *source* object. For each *target* object there are only videos. We want one image-only policy per target that succeeds on the target without losing the source, and we ask whether adapting the IDM to the target before labeling its videos helps.

Scope. The source is a $6 \times 6 \times 6$ cm red cube. Targets are (A) a blue cube of the same size (appearance), (B) a $9 \times 6 \times 6$ cm red block (geometry, same grasp width), (C) a $9 \times 7 \times 6$ cm red block (geometry and grasp width) and, optionally, (D) a blue version of C. Robot, cameras, lighting and task stay fixed. Human videos, large pretrained models and real robots are out of scope. [TODO: State which targets were completed; this draft has results for the source and A only.]

1.1 Constraints

Table 1 lists the requirements the design must satisfy (C4). R1–R3 define the supervision contract and hold by construction; R4 and R5 are the outcome targets; R6 is the real-time condition. R6 was restated when the control rate changed (Section 2.3): the policy is queried at every 10 Hz control step, so one call must finish within the 100 ms control period.

Table 1: Requirements (C4) and their status in this draft. Baseline numbers are from a single training seed and are preliminary.

ID	Requirement	Definition	Target	Status
R1	Supervision	Target data are RGB frames only	Zero target labels	By design; [TODO: confirm for the IDM pipeline]
R2	Label budget	Labeled source demonstrations, identical for every method	100; 300 if the source-only policy is below 80% in source validation	Pending: 300 demos reach 98% in validation; [TODO: 100-demo result and decision]
R3	Inputs	No simulator state in any learned model	Images only	Met by the policy (robot state zeroed); [TODO: IDM]
R4	Target	Success gain over the source-only policy on each target, as a fraction of the oracle's gain	$\geq 50\%$ of the gap	[TODO: R4 per target]
R5	Retention	Drop in source success after adding target data	≤ 10 points	[TODO: R5 per target]
R6	Real time	Policy inference per call, within one 10 Hz control step (IDM offline)	< 100 ms	Measured: 10 ms (95th percentile)

2 Methods

2.1 Problem formulation

An episode is a sequence of observations o_t (three RGB images) and actions $a_t \in \mathbb{R}^7$ (six arm joints and the gripper). The source dataset contains labeled transitions, $\mathcal{D}_S = \{(o_t, a_t, o_{t+1})\}$, from N_S demonstrations of the source object. Each target $T \in \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}\}$ has only videos, $\mathcal{D}_T = \{o_t\}$. Source and target share the robot, the cameras, the task and the expert's strategy; they differ in the object's appearance or geometry, so the observation distribution shifts and, for geometry changes, the action distribution may shift as well.

We seek, for each target, an image-only policy $\pi_T(a_{t:t+H} \mid o_t)$ that predicts a chunk of H future actions. Let $s(\pi, X)$ be the closed-loop success rate of policy π on object X. Three reference policies anchor the requirements: π_a trained on $\mathcal{D}_S$ only, π_d trained on $\mathcal{D}_S$ plus the target with its true actions (oracle), and the candidate π_T trained on $\mathcal{D}_S$ plus pseudo-labeled $\mathcal{D}_T$. R4 and R5 are

$$\text{R4:} \quad G_T = \frac{s(\pi_T, T) - s(\pi_a, T)}{s(\pi_d, T) - s(\pi_a, T)} \geq 0.5, \tag{1}$$

$$\text{R5:} \quad s(\pi_a, S) - s(\pi_T, S) \leq 0.10. \tag{2}$$

R4 is inconclusive for a target if the oracle does not exceed the source-only policy, $s(\pi_d, T) \leq s(\pi_a, T)$. True target actions are used only to train the oracle and to evaluate pseudo-label error; they never select a model (R1).

2.2 Datasets and resources

All data are generated by us in simulation; no licensing or ethics constraints apply.

Scene and task. We use OCBench's visual single-block task (`visual-block-cpu-lite-single-task1-v0`) through our fork of the simulator, which adds cube colours, a visible goal and reproducible initial states. A dark-gray 12 cm square marks the fixed goal. An episode succeeds when the whole object is inside the square, resting on the table, released and still for 1 s; the time limit is 200 control steps (20 s). The object starts uniformly in a 20×20 cm area beside the goal with its yaw fixed at 0, and the arm always starts from the same pose. The 12 cm side was chosen from the expert's own precision: 97% of its resting placements fit fully inside a 12 cm square, against 60% for 8 cm (200 episodes).

Table 2: Collections and seed ranges, fixed before any result. Demonstrations average 76 steps (7.6 s).

Collection	Seeds from	Episodes	Use
Source demonstrations	0	300	Training and validation, labeled
Source, held out	50,000	50	IDM test error only
Target A videos	100,000	200	Action-free; true actions: oracle, evaluation
Target B, C, D videos	200,000+	[TODO: 200 each]	Same as A
Validation rollouts	900,000	100	Checkpoint selection, R2 (source only)
Test rollouts	1,000,000	100	Final evaluation, every object

Observations and actions. Each control step (10 Hz) stores one 224×224 RGB image from each of the front, side and wrist cameras. OCBench actions are seven values in $[-1, 1]$: six arm-joint increments and one gripper-opening increment, each added to the current measured position. Simulator state (joint positions and velocities) is stored for replay, evaluation and label conversion, but is never an input of a learned model (R3).

Collections and seeds. Demonstrations come from OCBench's scripted expert. Every episode has its own seed, which fixes the initial state and the expert's random choices; failed attempts are discarded and recorded. Seed ranges do not overlap (Table 2), and the same seed gives the same initial object and arm state for every object, so all methods are evaluated from identical initial states. We verified that stored episodes replay exactly from their actions.

Deterministic against stochastic demonstrations. Stock OCBench demonstrations are deliberately varied: the cube has a random yaw, and the expert moves at a random speed and takes deliberate detours. We collected both this *stochastic* variant and a *deterministic* one (yaw fixed at 0, no detours, fixed speed), with the same seeds. The same unmodified ACT, trained on 300 source demonstrations with one seed, reaches 99% test success on the deterministic data and 53% on the stochastic data (Section 3). Evaluating the stochastic policy from fixed-yaw starts gives 71% in validation, so about 18 points of the gap come from the harder initial states and about 27 from the variability of the demonstrations. We therefore fixed the deterministic data as the project data for every method and object, before running any method that uses target videos: the research question concerns the transfer between objects, and a source-only policy that fails half the time on its own object would leave no room to measure retention (R5) or a meaningful oracle gap (R4). The stochastic collections are kept as a reference. The deterministic expert is not fully deterministic: it keeps OCBench's grasp randomness (a $\pm 90^\circ$ wrist mode in 30% of episodes, a pick hold in 40% and about 1 cm of contact jitter). Its success rate is 99.7% on the source (300 of 301 attempts) and 100% on target A.

Resources. Episodes are public on the Hugging Face Hub (`https://huggingface.co/datasets/Autobrik/ecse552-ocbench-block`); training downloads a pinned revision, and checkpoints are published in companion model repositories. A LeRobot copy [21] serves the online visualizer. Policies train on Modal with one NVIDIA A10G GPU, about 2 h per ACT run, with curves logged to Weights & Biases. Rendering and rollouts run on a local GPU. The code is at `https://github.com/BrikHMP18/ecse-552-project`.

2.3 Designed solution and implementation

The pipeline has three stages: (1) train an IDM on labeled source transitions, adapting it to the target with DANN; (2) label the target videos with the IDM; (3) train an image-only ACT policy on source demonstrations plus pseudo-labeled target videos. One policy is trained per target. IDM and policy share no weights.

Inverse dynamics model. The IDM f_θ predicts the action between two consecutive observations, $\hat{a}_t = f_\theta(o_t, o_{t+1})$. A convolutional encoder shared across cameras embeds both frames and their difference, which exposes motion and suppresses static appearance; an MLP regresses the action with a mean-squared-error loss on source transitions,

$$\mathcal{L}_{\text{act}}(\theta) = \mathbb{E}_{(o_t, a_t, o_{t+1}) \sim \mathcal{D}_S} \left\| f_\theta(o_t, o_{t+1}) - a_t \right\|_2^2. \tag{3}$$

The IDM is easier to learn from few labels than the policy because it does not need to model the expert's intent, only the effect of an action that already happened [9]. Its split takes blocks of ten source episodes, eight for training and two for validation. [TODO: IDM architecture (encoder, input resolution, output: absolute joint targets), optimizer, training length and source validation error.]

Domain-adversarial adaptation. Following DANN [14], a domain discriminator d_ϕ receives the IDM's features h_θ for source and target frame pairs and is trained with binary cross-entropy to tell them apart. A gradient reversal layer makes the encoder maximize the same loss, scaled by λ:

$$\min_{\theta} \max_{\phi} \quad \mathcal{L}_{\text{act}}(\theta) - \lambda\, \mathcal{L}_{\text{dom}}(\theta, \phi), \qquad \mathcal{L}_{\text{dom}} = \text{BCE}\big(d_\phi(h_\theta(o_t, o_{t+1})),\, y_{\text{dom}}\big). \tag{4}$$

The encoder is thus pushed towards features that predict source actions but do not reveal the domain. λ ramps up on a schedule fixed in advance, and $\lambda = 0$ gives the unadapted IDM of method (b). Only target *frames* enter this stage (R1). IDM checkpoints are selected by source validation error, never with target actions. [TODO: Discriminator architecture, λ schedule and final value, batch composition.]

Policy. The policy is the original ACT [3], run from the authors' code without modification through a thin adapter. ACT is a conditional VAE: a ResNet-18 backbone per camera feeds a transformer encoder (4 layers) and decoder (7 layers, 8 heads, hidden size 512, feed-forward size 3,200) that outputs a chunk of $H = 20$ actions (2 s); a separate encoder maps the action chunk to a 32-dimensional latent z during training, and $z = 0$ at test time. The loss is

$$\mathcal{L}_{\text{ACT}} = \big\|\hat{a}_{t:t+H} - a_{t:t+H}\big\|_1 + \beta\, D_{\text{KL}}\big(q(z \mid a_{t:t+H}, o_t) \,\|\, \mathcal{N}(0, I)\big), \qquad \beta = 10. \tag{5}$$

We keep the hyperparameters of the ACT repository (batch size 8, learning rate 10^{-5}, weight decay 10^{-4}, 2,000 epochs). The only change is that the robot-state input is set to zero, so the policy sees images only (R3).

Actions and execution. Policies are trained on absolute joint targets, computed from the stored increments and joint positions: $q_t^* = q_t + a_t \odot c$ for the arm, with c the simulator's per-joint scale, and the clipped opening for the gripper. A low-level controller converts each target back into the increment the simulator expects. At test time the policy is queried at every step and the overlapping chunks are combined by ACT's temporal ensembling: the executed target is $\sum_i w_i \hat{q}_t^{(i)} / \sum_i w_i$ with $w_i = \exp(-k\, i)$, $k = 0.01$, where $i = 0$ is the oldest prediction for step t.

Deviations from the proposal. Table 3 lists the decisions that differ from the proposal. Most follow from one finding: our first policy, a reimplementation of ACT with the proposal's settings (joint increments, open-loop chunks, stochastic data), did not exceed 49% validation success on the source cube (Section 4). We replaced it with the original ACT and adopted its conventions.

2.4 Evaluation

The protocol was fixed in the repository's configuration before any result.

Splits. Source demonstrations are split by episode into 80% training and 20% validation (ACT's own random split). Fifty held-out source demonstrations (seeds 50,000+) are used only for the IDM's test error. Closed-loop evaluation uses two disjoint seed ranges shared by all methods and objects: validation rollouts (seeds 900,000+) and test rollouts (seeds 1,000,000+).

Checkpoint selection. Each ACT run saves the checkpoint with the lowest validation loss (*best*) and the final one (*last*). We run 100 validation rollouts on the *source* object for each and keep the one with higher success, with ties going to *best*. Selection never uses the target's actions or the target's success, so R1 holds for model selection too. The R2 decision (100 or 300 demonstrations) uses the same validation rollouts.

Test. Each selected checkpoint is evaluated once: 100 test rollouts per object, from the same initial states for every method. Each method is trained with three seeds; we report the mean and standard

Table 3: Decisions that differ from the proposal.

Topic	Decision	Proposal	Reason
Control rate	10 Hz, at most 200 steps; chunk 20 (2 s)	50 Hz, chunk 25	One image per control step; the chunk keeps ACT's 2 s horizon
Actions	Absolute joint targets	Joint increments	ACT's convention; increments assume the state they were predicted from, so re-querying or averaging them fails
Execution	Temporal ensembling	Chunks open loop	Open loop gives 41% against 98% for the same policy (Table 6)
Initial state	Object yaw fixed at 0; 20×20 cm start area; fixed arm pose	OCBench defaults	As in ACT's simulated tasks; isolates the object shift
Expert	Deterministic: no detours, fixed speed	OCBench's, adapted per object	99% against 53% source success (Section 2.2)
Goal	12 cm square; inside, resting, released, still for 1 s	Within 4 cm	A visible goal and a strict success test
Policy	Original ACT code; checkpoint chosen by validation rollouts	Own ACT, fixed training steps	Our reimplementation underperformed; validation loss picks poor checkpoints
Split	80/20 train/validation by episode; test by rollouts on reserved seeds	80/10/10	Upstream ACT's split; success needs rollouts, not held-out demos
R6	< 100 ms per call	< 0.5 s per chunk	Temporal ensembling queries the policy every step

deviation across seeds and, for each success rate, the 95% Wilson score interval. For k successes in n rollouts, $\hat{p} = k/n$ and $z = 1.96$,

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}. \tag{6}$$

Intervals are necessary beyond sampling error: GPU rendering is not bit-exact, so rerunning the same seeds changes some outcomes (Section 4).

Metrics. (i) Success rate on each target and on the source, from which R4 (Eq. 1) and R5 (Eq. 2) follow. (ii) Inference latency per policy call during rollouts, after warm-up, reported as the 95th percentile (R6). (iii) IDM error per action component on held-out source transitions and, with the true target actions, pseudo-label error on the target videos; the second is a diagnostic only. (iv) A classification of failed rollouts: never lifted, placed just outside the goal, lifted and ended far, inside but not released.

2.5 Alternative solutions

All alternatives are run on our own data with the same ACT architecture, training budget and evaluation (Table 4). Methods (a) and (d) anchor R4: the source-only floor and the oracle with true target actions. Comparing (b) and (c) isolates the effect of adapting the IDM, since they differ only in λ. Method (e) tests whether target videos are needed at all: it trains on source data with hue, brightness, contrast and saturation jitter and small random crops [22]. We expect (e) to be competitive on target A, where only colour changes, and weaker on B and C, whose geometry and grasp width these augmentations do not reproduce. Optional method (f) moves the alignment from the IDM to the policy's encoder [18].

3 Results

All numbers in this section come from one training seed and are preliminary. [TODO: Replace with three seeds (mean, standard deviation) once the runs finish.]

Table 4: Methods compared. All use the same source demonstrations and ACT policy.

	Method	Target data	Target labels
(a)	Source only	None	None
(b)	Unadapted IDM ($\lambda = 0$)	Videos	IDM pseudo-labels
(c)	DANN-adapted IDM (ours)	Videos	IDM pseudo-labels
(d)	Oracle	Demonstrations	True actions
(e)	Source only with augmentation	None	None
(f)	(b) with policy-encoder alignment (optional)	Videos	IDM pseudo-labels

Table 5: Baseline (a): original ACT, 300 source demonstrations, one seed. Validation and test success over 100 rollouts each; 95% Wilson intervals in brackets. The robot-state run breaks R3 and is a reference only.

		Validation (source)		Test		Latency
Data	Input	best	last	Source	Target A	p95
Stochastic	Images	40%	53%	53% [43, 62]	0% [0, 4]	10 ms
Deterministic (project)	Images	61%	98%	**99%** [95, 100]	**7%** [3, 14]	10 ms
Deterministic	Images + state	44%	66%	69% [59, 77]	1% [0, 5]	10 ms

3.1 Source-only baseline (a)

Table 5 reports the original ACT trained on 300 source demonstrations. On the project data the image-only policy reaches 99% on the source cube, which meets R3 and the 80% level of R2 at 300 demonstrations. Changing only the cube's colour breaks it: 7% on target A. The floor of R4 is therefore near zero, and the methods that use A's videos have a large gap to close. Inference takes 10 ms per call (95th percentile), a tenth of the R6 budget.

In all three runs the final checkpoint was selected: the checkpoint with the lowest validation loss was always worse in rollouts (Figure 2). On the source cube, the selected deterministic policy fails once in 100 validation rollouts (cube never lifted); the stochastic policy fails mostly by never lifting the cube (38) or placing it just outside the goal (11). Adding the robot state did not help (69%); its success varies widely between nearby checkpoints, so one seed does not support a claim. [TODO: R2: validation success with 100 demonstrations and the resulting budget.]

3.2 Execution ablation

Table 6 compares four ways of executing the same policy's 20-action chunks. Temporal ensembling is essential: executing each chunk open loop, as the proposal planned, drops success from 98% to 41%, and re-querying more often without averaging is worse.

3.3 Inverse dynamics model

[TODO: Table: IDM error per action component on held-out source transitions, and pseudo-label error on each target, for $\lambda = 0$ (b) and DANN (c); three seeds.]

3.4 Comparison with alternatives

Table 7 will hold the comparison on the C4 constraints (C9, C10).

[TODO: Text: which methods meet R4 and R5 on each target; (b) against (c); (e) against (c); latency for every method (R6); negative transfer if alignment hurts.]

4 Discussion and Conclusions

Were the constraints met? So far R3 and R6 are met by the baseline, with a wide margin for R6, and R2's 80% level is met at 300 demonstrations. [TODO: R1, R2 decision, R4 and R5 with the final results.]

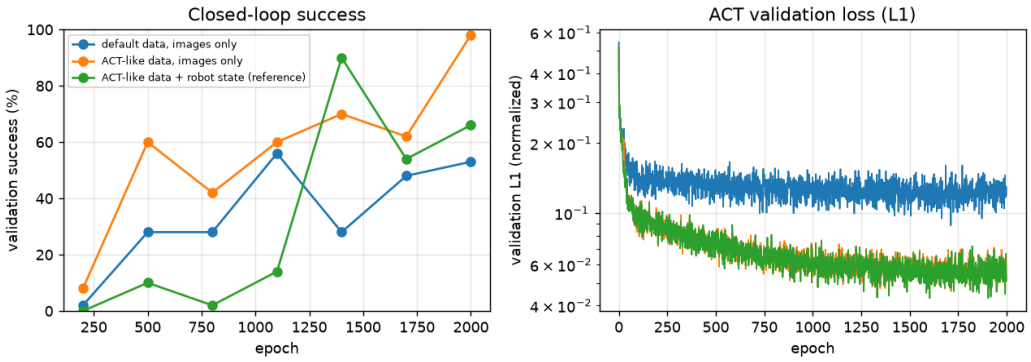


Figure 2: Validation success per epoch (50 rollouts; 100 at the last epoch) and ACT's validation loss for the three baseline runs. The loss is smooth while success varies between nearby checkpoints.

Table 6: Execution of the 20-action chunks: deterministic-data policy, 100 source validation rollouts, one seed.

Execution	Success
Temporal ensembling, $k = 0.01$ (used everywhere)	**98%**
Open loop: whole chunk, then re-query (proposal)	41%
Re-query every 10 steps	32%
Re-query every step, first action only	1%

Demonstration variability dominates. The same code and data budget give 99% or 53% depending on how the demonstrations were generated. The ACT paper reports the same direction between scripted and human demonstrations [3]. For this project it means that the quality of the source data, not the policy architecture, decided whether the study was possible.

Validation loss does not select good policies. In every run, with both our reimplementation and the original ACT, the checkpoint with the lowest validation loss succeeded less than the final one (for example 61% against 98%). The loss curve is smooth while success varies between nearby checkpoints. Selection therefore needs closed-loop rollouts, which is cheap in simulation and costly on a real robot. [TODO: Cite prior reports of this effect (not yet in the bibliography).]

What went wrong. We first wrote our own image-only ACT with the proposal's settings: joint increments, chunks of 10 executed open loop, stochastic demonstrations. With 100, 200 and 300 demonstrations it reached 23%, 49% and 31% validation success (one seed each); even from training initial states it succeeded only 60–70% of the time, and most failures were missed grasps. Temporal ensembling made it worse (21% and 14% for 200 and 300 demonstrations), because averaging increments predicted from different states is not well defined. Comparing against the original ACT pointed to the action representation, the execution scheme and the demonstrations, not to training length. Running the original code with absolute targets and deterministic demonstrations solved the source task. The proposal's plan to execute chunks open loop would have cost 57 points.

Limitations.

- **Simplified task.** The object's yaw, the arm's start pose and the expert's speed are fixed, and the start area is 20×20 cm. The stochastic reference (53%) shows that the baseline does not yet handle stock OCBench variability.
- **Simulation only.** Target videos come from the same simulator and expert as the source, so the only shift is the object. Real or generated videos add shifts in appearance and dynamics that we do not study.
- **Evaluation noise.** GPU rendering is not bit-exact: two renders of the same state can differ by one intensity level in a few pixels. With our earlier reimplementation this flipped 11 of 40

Table 7: Test success (mean ± standard deviation over three seeds, 100 rollouts each), R4 gap closed and R5 retention drop. [TODO: Fill; add targets B and C.]

Method	Source	Target A	R4: G_A	R5: drop
(a) Source only	[TODO:]	[TODO:]	0 (ref.)	0 (ref.)
(b) Unadapted IDM	[TODO:]	[TODO:]	[TODO:]	[TODO:]
(c) DANN-adapted IDM	[TODO:]	[TODO:]	[TODO:]	[TODO:]
(d) Oracle	[TODO:]	[TODO:]	1 (ref.)	[TODO:]
(e) Augmentation	[TODO:]	[TODO:]	[TODO:]	[TODO:]

episode outcomes when the same seeds were rerun. We report intervals and never compare single reruns.

- **Seeds.** Current numbers use one training seed, and success varies between checkpoints of one run. [TODO: Update after three seeds.]
- **Adversarial alignment.** DANN aligns marginal feature distributions; it does not guarantee that actions remain readable from the aligned features [19], and it is weaker for regression than for classification [20]. Per-component IDM errors test this. [TODO: Outcome.]
- **Operations.** The original ACT code cannot resume, so a preempted cloud run restarts from zero.

Advantages. The design needs no target labels, no simulator state and no pretrained model; the IDM runs offline, so the deployed policy costs the same as the baseline (10 ms per call). Every comparison shares data, seeds, initial states, policy code and selection rule, and the protocol was fixed before the results. Data, checkpoints and code are public. [TODO: Advantages supported by the final results.]

Conclusion. [TODO: Answer the research question: do action-free videos, labeled by an adapted IDM, close the gap to the oracle on each target while retaining the source?]

Contributions

All members collaborated on and reviewed the report. [TODO: Update to what each member did.]

Member	Role	Tasks
Brik Meza	Data and evaluation	Simulation, expert demos, datasets, splits, rollouts, metrics
Myriam Dardour	Adversarial adaptation	Discriminator, adversarial training, tuning, experiments
Zheng Ye Zhang	Inverse dynamics model	IDM architecture and training, pseudo-labeling, error analysis
Gonzalo Millán García	Baselines and alternatives	Policy training, baselines, alternative methods, comparisons

Generative AI disclosure

We used ChatGPT and Codex (OpenAI) and Claude and Claude Code (Anthropic), following McGill's guidelines on generative AI in research. They assisted with (i) the literature search and bibliography formatting; (ii) document structure and draft prose of the proposal and this report; (iii) software: simulator changes in our OCBench fork, seeded data collection and replay checks, dataset publishing, the training and evaluation infrastructure, a first ACT reimplementation and its diagnostics, the adapter that runs the original ACT, and tests; and (iv) running and summarizing the baseline experiments that a team member approved. We used them to cover more engineering than a four-person course project otherwise allows, so that the team's time went to design decisions and analysis. The team directed the work, made the design decisions (scene, data sizes, protocol, ACT settings) and approved every paid run. Verification: automated tests, exact replay of stored episodes, end-to-end training and evaluation tests, and an independent review of the ACT adapter against the upstream code; every

reported number comes from logged runs. The team revised the text and checked claims and citations against the sources. [TODO: Update from docs/ai-use-log.md before submission, including the IDM and DANN work.]

References

[1] Seohong Park and Sergey Levine. Behavioral cloning mystery. arXiv preprint arXiv:2610.07056, 2026. URL https://arxiv.org/abs/2610.07056. OCBench: https://github.com/seohongpark/ocbench.

[2] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[3] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.016. URL https://roboticsproceedings.org/rss19/p016.html. arXiv:2304.13705.

[4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023. https://diffusion-policy.cs.columbia.edu/.

[5] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2403.12945.

[6] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, et al. DreamGen: Unlocking generalization in robot learning through video world models. In *Conference on Robot Learning (CoRL)*, volume 305 of *Proceedings of Machine Learning Research*, pages 5170–5194, 2025. URL https://proceedings.mlr.press/v305/jang25a.html.

[7] Faraz Torabi, Garrett Warnell, and Peter Stone. Behavioral cloning from observation. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 4950–4957, 2018. URL https://arxiv.org/abs/1805.01954.

[8] Bowen Baker, Ilge Akkaya, Peter Zhokov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video PreTraining (VPT): Learning to act by watching unlabeled online videos. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9c7008aff45b5d8f0973b23e1a22ada0-Abstract-Conference.html.

[9] Sacha Morin, Moonsub Byeon, Alexia Jolicoeur-Martineau, and Sébastien Lachapelle. On the sample efficiency of inverse dynamics models for semi-supervised imitation learning. In *International Conference on Machine Learning (ICML)*, 2026. URL https://arxiv.org/abs/2602.02762.

[10] Moo Jin Kim, Jiajun Wu, and Chelsea Finn. Giving robots a hand: Learning generalizable manipulation with eye-in-hand human video demonstrations, 2023. URL https://arxiv.org/abs/2307.05959.

[11] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejune Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gao, Luke Zettlemoyer, Dieter Fox, and Minjoon Seo. Latent action pretraining from videos. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.11758.

[12] Jeremy A. Collins, Loránd Cheng, Kunal Aneja, Albert Wilcox, Benjamin Joffe, and Animesh Garg. AMPLIFY: Actionless motion priors for robot learning from videos, 2025. URL https://arxiv.org/abs/2506.14198.

[13] Alexander Nikulin, Ilya Zisman, Denis Tarasov, Nikita Lyubaykin, Andrei Polubarov, Igor Kiselev, and Vladislav Kurenkov. Latent action learning requires supervision in the presence of distractors. In *International Conference on Machine Learning (ICML)*, 2025.

[14] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario March, and Victor Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35, 2016. URL https://www.jmlr.org/papers/v17/15-239.html.

[15] Eric Tzeng, Judy Hoffman, Kate Saenko, and Trevor Darrell. Adversarial discriminative domain adaptation. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2017. URL https://arxiv.org/abs/1702.05464.

[16] Mingsheng Long, Zhangjie Cao, Jianmin Wang, and Michael I. Jordan. Conditional adversarial domain adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[17] Shuo Cheng, Liqian Ma, Zhenyang Chen, Ajay Mandlekar, Caelan Reed Garrett, and Danfei Xu. Generalizable domain adaptation for sim-and-real policy co-training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL https://arxiv.org/abs/2509.18631.

[18] Yasuhiro Kato, Thomas Westfechtel, Jen-Yen Chang, Naoki Morihira, Akinobu Hayashi, Tatsuya Harada, and Takayuki Osa. Unsupervised domain adaptation for robust imitation learning under visual perturbations. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9703–9710, 2026. doi: 10.1109/ICRA57385.2026.11696866.

[19] Han Zhao, Remi Tachet des Combes, Kun Zhang, and Geoffrey J. Gordon. On learning invariant representations for domain adaptation. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 7523–7532, 2019. URL https://proceedings.mlr.press/v97/zhao19a.html.

[20] Ismail Nejjar, Qin Wang, and Olga Fink. DARE-GRAM: Unsupervised domain adaptation regression by aligning inverse gram matrices. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11744–11754, 2023.

[21] Remi Cadene et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. GitHub, 2024.

[22] Michael Laskin, Kimin Lee, Adam Stooke, Lerrel Pinto, Pieter Abbeel, and Aravind Srinivas. Reinforcement learning with augmented data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.